УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 7

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ РАБОТЫ С КОРТЕЖАМИ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейших программ работы с кортежами

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Кортежи в Python

Кортежи, как и списки, являются упорядоченными последовательностями элементов. Они во многом аналогичны спискам, но имеют одно очень важное отличие – изменить кортеж нельзя. Можно сказать, что кортеж – это список, доступный только для чтения.

Создать кортеж можно следующими способами:

– с помощью функции **tuple** ([<Последовательность>]). Функция позволяет преобразовать любую последовательность в кортеж. Если параметр не указан, то создается пустой кортеж.

Пример:

```
print(tuple ()) #  Создаем пустой кортеж
( )
print(tuple ("String")) #  Преобразуем строку в кортеж
('S', 't', 'r', 'i', 'n', 'g')
print(tuple ( [1,   2,   3,   4,   5] ))  #  Преобразуем список в  кортеж
(1,   2,   3,   4,   5)
```

– указав все элементы через запятую внутри круглых скобок (или без скобок):

```
tl  = () #  Создаем пустой кортеж
t2 = (5,) # Создаем кортеж из одного элемента
t3  = (l, "str", (3,   4) )  #  Кортеж из трех элементов
t4  = 1, "str", (3, 4)       #  Кортеж из трех элементов
print( t1, t2, t3, t4)
( ( ), (5,), (1, 'str', (3, 4)), (1, 'str', (3, 4)) )
```

Обратите особое внимание на вторую строку примера. Чтобы создать кортеж из одного элемента, необходимо в конце указать запятую. Именно запятые

формируют кортеж, а не круглые скобки. Если внутри круглых скобок нет запятых, то будет создан объект другого типа.

Пример:

```
t = (5)
print(type(t)) # Получили число, а не кортеж!
<class 'int'>
t = ("str")
print(type(t)) # Получили строку, а не кортеж!
<class 'str'>
```

Четвертая строка в исходном примере также доказывает, что не скобки формируют кортеж, а запятые. Помните, что любое выражение в языке Python можно заключить в круглые скобки, а чтобы получить кортеж, необходимо указать запятые.

Позиция элемента в кортеже задается индексам. Обратите внимание на то, что нумерация элементов кортежа (как и списка) начинается с о, а не с 1. Как и все последовательности, кортежи поддерживают обращение к элементу по индексу, получение среза, конкатенацию (оператор +), повторение (оператор *), проверку на вхождение (оператор in) и невхождение (оператор not in).

Пример:

```
t = (1, 2, 3, 4, 5, 6, 7, 8, 9)
print(t[0]) # Получаем значение первого элемента кортежа
1
print( t[::-l]) # Изменяем порядок следования элементов
(9, 8, 7, 6, 5, 4, 3, 2, 1)
print(t[2:5]) # Получаем срез
(3, 4, 5)
print(8 in t, 0 in t) # Проверка на вхождение
(True, False)
print( (1, 2, 3) * 3) # Повторение
(1, 2, 3, 1, 2, 3, 1, 2, 3)
print( (1, 2, 3) + (4, 5, 6)) # Конкатенация
(1, 2, 3, 4, 5, 6)
```

Кортежи, как уже неоднократно отмечалось, относятся к неизменяемым типам данных. Иными словами, можно получить элемент по индексу, но изменить его нельзя:

```
t  = (1, 2, 3) #  Создаем кортеж
print(t [0]) #  Получаем элемент  по индексу
1
print( t[0] = 50 )# Изменить элемент по индексу нельзя!
Traceback (most recent call last):
   File "<pyshell#95>", line 1, in <module>
     t[0] =50 # Изменить элемент по индексу нельзя!
TypeError: 'tuple' object does not support item assignment
```

Кортежи поддерживают уже знакомые по спискам функции 1en (), min (), max (), методы index () и count ().

Примеры:

```
t = (1, 2, 3)  # Создаем кортеж
print(len(t)) # Получаем количество элементов
3
t = (1, 2, 1, 2, 1)
print(t.index (1) , t.index (2)) # Ищем элементы в кортеже
(0, 1)
```

Кортеж (tuple) представляет последовательность элементов, которая во многом похожа на список за тем исключением, что кортеж является неизменяемым (immutable) типом. Поэтому мы не можем добавлять или удалять элементы в кортеже, изменять его.

Для создания кортежа используются круглые скобки, в которые помещаются его значения, разделенные запятыми:

```
tom = ("Tom", 23)
print(tom)     # ("Tom", 23)
```

Также для определения кортежа мы можем просто перечислить значения через запятую без применения скобок:

```
tom = "Tom", 23
print(tom)     # ("Tom", 23)
```

Если вдруг кортеж состоит из одного элемента, то после единственного элемента кортежа необходимо поставить запятую:

```
tom = ("Tom",)
```

Для создания кортежа из другого набора элементов, например, из списка, можно передать список в функцию tuple(), которая возвратит кортеж:

```
data = ["Tom", 37, "Google"]
tom = tuple(data)
print(tom)      # ("Tom", 37, "Google")
```

С помощью встроенной функции len() можно получить длину кортежа:

```
tom = ("Tom", 37, "Google")
print(len(tom))     # 3
```

4.2 Обращение к элементам кортежа

Обращение к элементам в кортеже происходит также, как и в списке, по индексу. Индексация начинается также с нуля при получении элементов с начала списка и с -1 при получении элементов с конца списка:

```
tom = ("Tom", 37, "Google", "software developer")
print(tom[0])       # Tom
print(tom[1])       # 37
print(tom[-1])      # software developer
```

Но так как кортеж - неизменяемый тип (immutable), то мы не сможем изменить его элементы. То есть следующая запись работать не будет:

```
tom[1] = "Tim"
```

При необходимости мы можем разложить кортеж на отдельные переменные:

```
name, age, company, position = ("Tom", 37, "Google", "software developer")
print(name)        # Tom
print(age)         # 37
print(position)    # software developer
print(company)     # Google
```

4.3 Получение подкортежей

Как и в списках, можно получить часть кортежа в виде другого кортежа

```
tom = ("Tom", 37, "Google", "software developer")
# получем подкортеж с 1 по 3 элемента (не включая)
print(tom[1:3])     # (37, "Google")
# получем подкортеж с 0 по 3 элемента (не включая)
print(tom[:3])     # ("Tom", 37, "Google")
# получем подкортеж с 1 по послдений элемент
print(tom[1:])     # (37, "Google", "software developer")
```

4.4 Кортеж как параметр и результат функций

Особенно удобно использовать кортежи, когда необходимо возвратить из функции сразу несколько значений. Когда функция возвращает несколько значений, фактически она возвращает в кортеж:

```
def get_user():
    name = "Tom"
    age = 22
    company = "Google"
    return name, age, company
user = get_user()
print(user)     # ("Tom", 37, "Google")
```

При передаче кортежа в функцию с помощью оператора * его можно разложить на отдельные значения, которые передаются параметрам функции:

```
def print_person(name, age, company):
    print(f"Name: {name}  Age: {age}  Company: {company}")
tom = ("Tom", 22)
print_person(*tom, "Microsoft")    # Name: Tom  Age: 22  Company: Microsoft
bob = ("Bob", 41, "Apple")
print_person(*bob)      # Name: Bob  Age: 41  Company: Apple
```

4.5 Перебор кортежей

Для перебора кортежа можно использовать стандартные циклы for и while. С помощью цикла for:

```
tom = ("Tom", 22, "Google")
for item in tom:
    print(item)
```

С помощью цикла while:

```
tom = ("Tom", 22, "Google")
i = 0
while i < len(tom):
    print(tom[i])
    i += 1
```

4.6 Проверка наличия значения

Как для списка с помощью выражения элемент in кортеж можно проверить наличие элемента в кортеже:

```
user = ("Tom", 22, "Google")
name = "Tom"
if name in user:
    print("Пользователя зовут Tom")
else:
    print("Пользователь имеет другое имя")
```

4.7 Диапазоны

Диапазоны у как следует из самого их названия — это диапазоны целых чисел с заданными начальным и конечным значением и шагом (промежутком между соседними числами). Как и списки, кортежи и множества, диапазоны представляют собой последовательности и, подобно кортежам, являются неизменяемыми.

Важнейшим преимуществом диапазонов перед другими видами последовательностей является их компактность - вне зависимости от количества входящих в него элементов-чисел, диапазон всегда отнимает один и тот же объем оперативной памяти. Однако в диапазон могут входить лишь последовательно стоящие друг за другом числа - сформировать диапазон на основе произвольного набора чисел или данных другого типа, даже чисел с плавающей точкой, невозможно.

Диапазоны чаще всего используются для проверки вхождения значения в какой-либо интервал и для организации циклов.

Для создания диапазона применяется функция **range ()**:

range ([<Начало>,] <Конец> [, <Шаг>])

Первый параметр задает начальное значение - если он не указан, используется значение 0. Во втором параметре указывается конечное значение. Следует заметить, что это значение не входит в возвращаемый диапазон. Если параметр **<Шаг>** не указан, то используется значение 1.

Примеры:

```
r = range(1, 10)
for  i  in r: print (i, end = " ")
1 2 3 4 5 6 7 8 9
r = range (10, 110, 10)
```

```
for i in r: print (i, end = " ")
10 20 30 40 50 60 70 80 90 100
r = range (10, 1, -1)
for i in r: print (i, end = " ")
10 9 8 7 6 5 4 3 2
```

Преобразовать диапазон в список, кортеж, обычное или неизменяемое множество можно с помощью функций **list (), tuple (), set ()** или **frozenset ()** соответственно:

```
print(list(range(1, 10))) # Преобразуем в список
[1, 2, 3, 4, 5, 6, 7, 8, 9]
print(tuple(range(1, 10))) # Преобразуем в кортеж
(1, 2, 3, 4, 5, 6, 7, 8, 9)
print(set(range(1, 10))) # Преобразуем в множество
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

Диапазоны поддерживают доступ к элементу по индексу, получение среза (в результате возвращается также диапазон), проверку на вхождение и невхождение, функции **len(), min(), max()**, методы **index()** и **count()**.

Примеры:

```
r = range(1, 10)
print(r[2], r[-1])
(3, 9)
print(r[2:4])
range(3, 5)
print(2 in r, 12 in r)
(True,   False)
print(3 not in r, 13 not in r)
(False, True)
print(len(r), min(r), max(r))
(9, 1, 9)
print(r.index(4), r.count(4))
(3, 1)
```

4.8 Пример решения задачи на языке программирования Python

Задача. В кортеже целых чисел вычислите произведение отрицательных элементов, имеющих нечетные индексы.

Разработка алгоритма решения задачи представлена на рисунке 1.

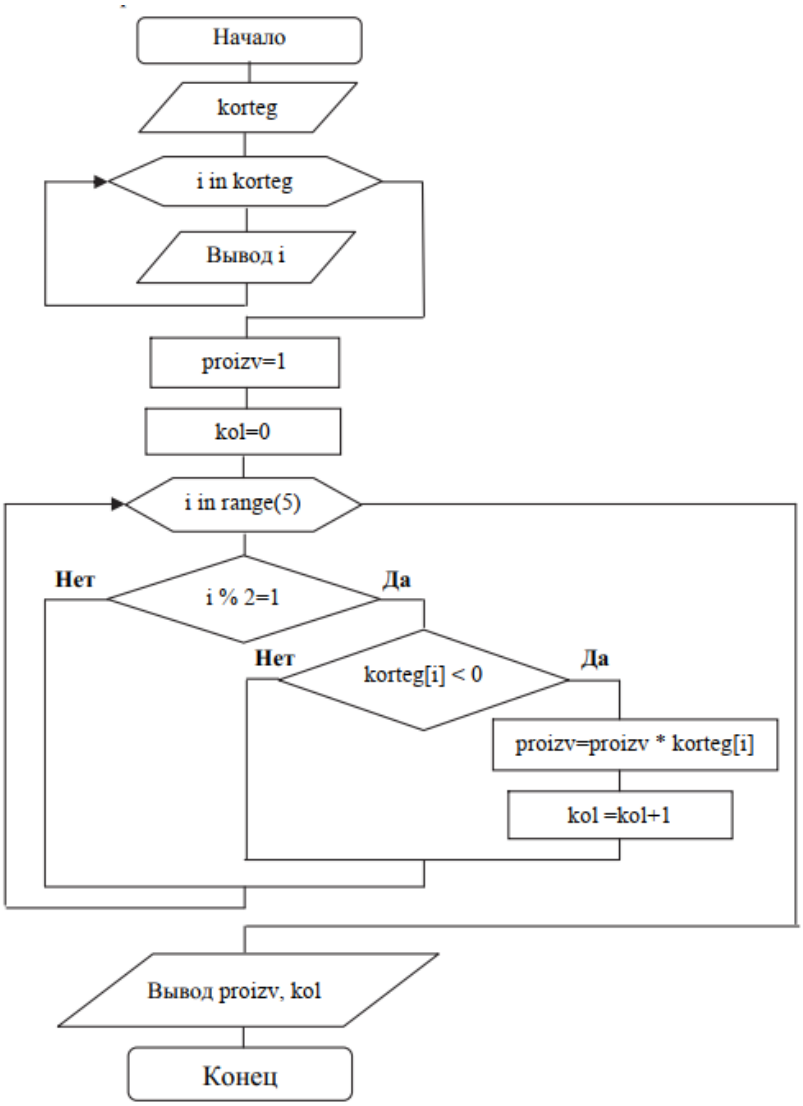


Рисунок 1 – Алгоритм решения задачи

Решение задачи:

```
korteg=(13, -2, -23, -14, -5)
print("\nКортеж")
for i in korteg:
   print(i, end=" ")
kol=0
proizv=1
for i in range(5):
   if i%2==1: #Перебор элементов с нечетными индексами
      if korteg[i]<0:
         proizv = proizv*korteg[i] #Нахождение произведения отрицательных элементов
         kol = kol+1 #Подсчет количества
print("\nПроизведение отрицательных элементов, имеющих нечетные индексы в кортеже, равно ", proizv)
print("\nИх количество равно ", kol)
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 В кортеже целых чисел вычислите произведение отрицательных элементов, имеющих нечетные индексы.

2 В кортеже целых чисел вычислите среднее арифметическое квадратов положительных элементов.

3 В кортеже целых чисел определите число инверсий. Инверсией называется пара элементов, в которой большее число расположено слева от меньшего.

4 В кортеже из десяти целых чисел, сформированном случайным образом, найдите максимальный и минимальный элементы, а также выполните их обмен.

5 В кортеже вещественных чисел выделите положительные элементы в отдельный список. Определите их количество и найдите произведение элементов полученного списка.

6 В кортеже из N элементов, сформированном случайными целыми числами разного знака, сформируйте другой кортеж, содержащий элементы, модуль которых больше заданного значения C.

7 В кортеже целых чисел вычислите произведение отрицательных элементов, имеющих нечетные индексы.

8 В кортеже целых чисел A(10) определите максимальный и минимальный элементы среди положительных нечетных элементов.

9 В кортеже целых чисел выполните обмен соседних элементов, расположенных на четных и нечетных позициях.

10 В кортеже целых чисел найдите индекс последнего отрицательного элемента.

11 В кортеже целых чисел A(10) определите, имеется ли хотя бы одна пара одинаковых по значению элементов.

12 В кортеже целых чисел сформируйте два списка: список четных элементов B и список нечетных элементов C.

13 В кортеже целых чисел определите среднее арифметическое элементов, для которых выполняется условие |a[i]| > C (значение C вводится с клавиатуры).

14 В кортеже целых чисел определите количество элементов, значения которых больше среднего арифметического всех элементов кортежа.

15 В кортеже целых чисел найдите сумму элементов, расположенных между первым максимальным и первым минимальным элементами (не включая их).

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Найдите сумму положительных элементов кортежа, расположенных на чётных позициях.

2 Определите количество нулевых элементов в кортеже и выведите их индексы.

3 Вычислите разность между максимальным и минимальным элементами кортежа.

4 Найдите сумму квадратов всех нечётных чисел, входящих в кортеж.

5 Определите количество отрицательных элементов кортежа, модуль которых больше 10.

6 Сформируйте новый кортеж, содержащий только элементы исходного кортежа, делящиеся на 3 без остатка.

7 Найдите среднее арифметическое всех элементов кортежа и выведите элементы, большие этого среднего.

8 Определите индекс первого отрицательного элемента кортежа. Если отрицательных элементов нет — выведите соответствующее сообщение.

9 Найдите произведение элементов, расположенных между минимальным и максимальным элементами (не включая их).

10 Сформируйте новый кортеж из элементов исходного, исключив все отрицательные значения.

11 Найдите сумму модулей элементов кортежа, стоящих на позициях, кратных 3.

12 Определите количество пар соседних элементов кортежа, у которых сумма положительна.

13 Выведите индексы всех элементов кортежа, которые больше среднего арифметического всех элементов.

14 Определите количество элементов кортежа, значения которых встречаются в нём более одного раза.

15 Найдите наибольший положительный и наименьший отрицательный элементы кортежа и вычислите их сумму.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Приведите синтаксис объявления кортежей в языке программирования Python.

7.2 Опишите как можно реализовать в программе срез кортежа в языке программирования Python?

7.3 Опишите как осуществляется обмен значений элементов кортежа в языке программирования Python?

7.4 Перечислите и опишите операторы и методы для работы со множествами в языке Python.

7.5 Перечислите и опишите операторы сравнения множеств в языке программирования Python.

7.6 Перечислите и опишите методы для работы со множествами в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.